



Agentes, equipos y herramientas

Dirigido a: **Administradores de tenant y operadores de la plataforma**

Este manual explica cómo gestionar el **capital agéntico** de tu tenant: los agentes de IA que ejecutan las tareas, los agentes humanos que representan a personas asignables, los equipos que agrupan agentes para trabajar de forma cooperativa, y el catálogo de herramientas (tools) que define qué puede hacer cada agente.

Recorreremos en profundidad cinco áreas del panel de administración: el catálogo de agentes IA (/admin/agents) y la ficha de detalle de cada agente (con su Hub de Capacidad, su persona, sus knowledge bases, sus tools y sus skills), los agentes humanos (/admin/human-agents), los equipos (/admin/teams) con su detalle y la adopción de equipos built-in, y el catálogo de tools (/admin/tools). En cada pantalla verás qué información se muestra, para qué sirve cada control y qué acciones de alta y edición están disponibles según tu rol.

El modelo mental que vertebra toda la sección de agentes es el de las **cuatro vías de capacidad**: **SABER** (qué conocimiento tiene asignado: knowledge bases), **RECORDAR** (qué memoria persiste entre ejecuciones y con qué alcance), **SER** (su persona: proveedor LLM, modelo, temperatura y system prompt) y **HACER** (qué tools puede invocar). Cada una se configura en su sección de la ficha del agente y el Hub de Capacidad las resume con su estado real.

La mayoría de acciones de creación y edición requieren rol **administrador del tenant**; los usuarios sin ese rol pueden explorar y consultar, pero verán ocultos los botones de gestión. Los elementos **built-in** (mantenidos por la plataforma) son siempre de solo lectura: la vía para personalizarlos es crear una copia (fork del agente, adopción del equipo).

Cerramos el manual con tres pasos dedicados a las **tres vías para ampliar lo que un agente puede HACER**: los tipos de implementación de las tools personalizadas (con un ejemplo real en Python), las skills (fragmentos de prompt que crean hábitos compartibles) y los servidores

Generado · 2026-07-18

Idioma · Español

Versión · Producción

Confidencial · Comité de Dirección

conecta, la tool ejecuta, la skill instruye.

Contenido

- 01** Catálogo de agentes IA (built-in)

- 02** Plantillas del tenant y agentes locales de proyecto

- 03** Crear un nuevo agente IA

- 04** Ficha de detalle de un agente

- 05** Hub de Capacidad: SABER / RECORDAR / SER / HACER

- 06** Persona del agente (SER): modelo y prompt efectivo

- 07** Knowledge Bases del agente (SABER)

- 08** Tools y skills del agente (HACER)

- 09** Personalizar un agente (crear copia / fork)

- 10** Agentes humanos

- 11** Crear o editar un agente humano

- 12** Plantillas globales de agentes humanos

- 13** Equipos

- 14** Detalle de un equipo

- 15** Adoptar un equipo built-in

- 16** Catálogo de tools (herramientas)

- 17** Crear o editar una tool personalizada

- 18** Tipos de implementación de una tool: quién la ejecuta cuando el agente la invoca

- 19** Skills en profundidad: el prompt_fragment que crea hábitos

- 20** MCP servers del proyecto: conectar, importar y usar tools externas

1 Catálogo de agentes IA (built-in)

Esta es la pantalla principal de agentes de IA, accesible desde **Inicio > Agentes**. Un **agente** es una entidad con un **rol** (project_manager, architect, backend_dev, frontend_dev, qa, reviewer, leader, worker, specialist, researcher, devops, security o technical_writer) y una persona configurada: proveedor LLM, modelo y temperatura más su system prompt bilingüe (ES/EN).

El catálogo se organiza en tres pestañas según el ámbito (scope) del agente: **Built-in** (agentes mantenidos por la plataforma, que ves seleccionada en esta captura), **Plantillas del Tenant** (plantillas reutilizables propias de tu organización) y **Locales del Proyecto** (agentes específicos de un proyecto, normalmente forkados de un built-in o plantilla). Cada pestaña muestra entre paréntesis cuántos agentes contiene.

Encima de las pestañas hay dos filtros combinables: **Pertenencia** (Todos / En equipo / Sin equipo) y **Equipo** (lista con los equipos del tenant), útiles para responder preguntas como «¿qué agentes forman parte del equipo X?» o «¿qué agentes están sueltos, sin equipo?».

Cada agente aparece como una tarjeta con su nombre, una insignia de ámbito, su rol, su descripción, un fragmento del system prompt en el idioma activo de la interfaz y las insignias de los **equipos** a los que pertenece (o la marca «Sin equipo»). Si el agente fue forkado de otro, se indica con la nota «Forked from another agent». Al pulsar una tarjeta accedes a su ficha de detalle, que cubrimos unos pasos más adelante.

Si tienes rol de administrador del tenant verás el botón **Nuevo agente** en la esquina superior derecha, que abre el formulario de alta (lo cubrimos en el paso 3).

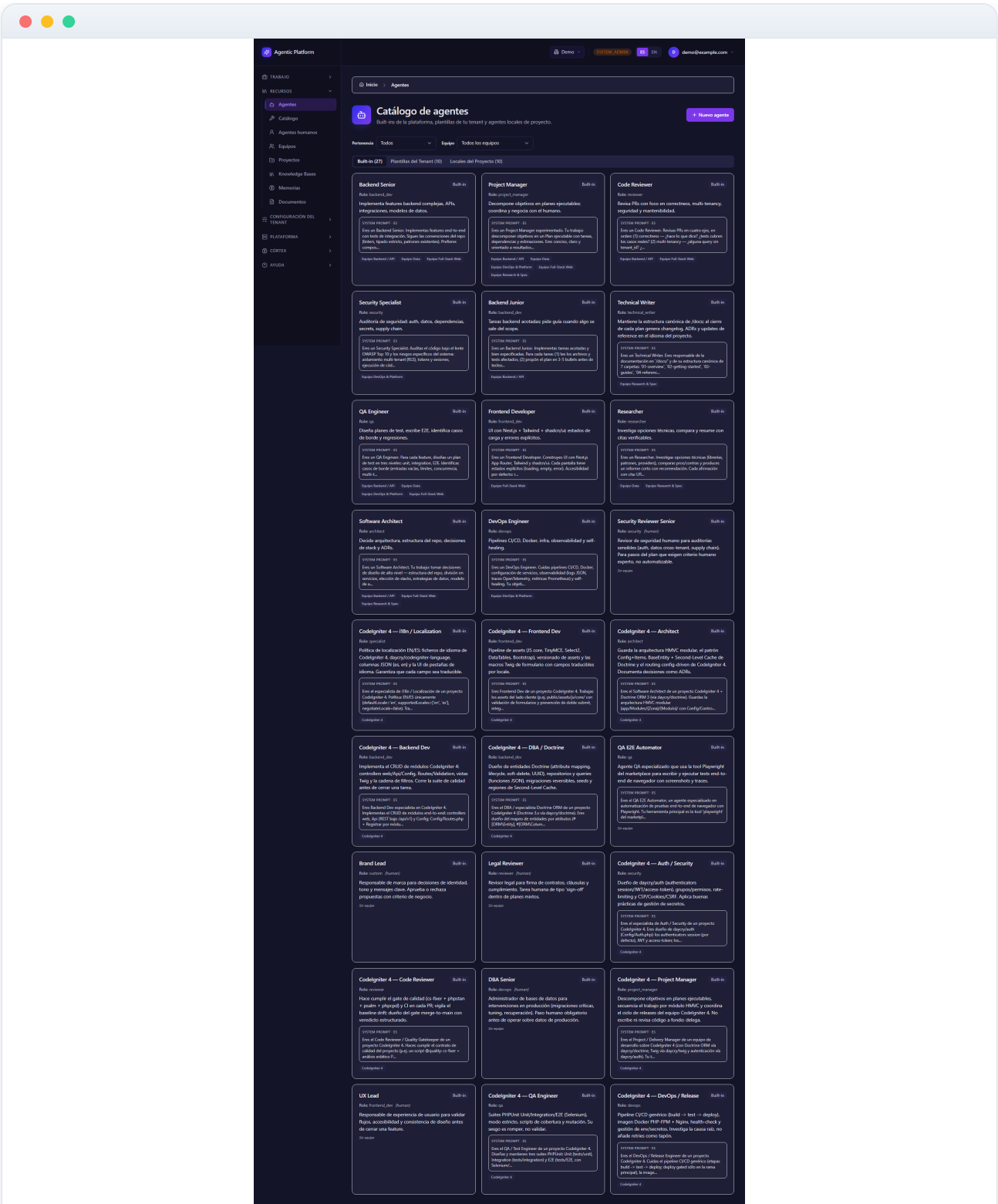


Figura 03.1 — Catálogo de agentes IA (built-in)

2 Plantillas del tenant y agentes locales de proyecto

Las otras dos pestañas del catálogo separan los agentes que crea tu organización. En **Plantillas del Tenant** (la que mostramos en esta captura) viven los agentes reutilizables en todos los proyectos del tenant: plantillas que defines una vez y reaprovechas al planificar. Es el ámbito adecuado para «vuestro backend dev estándar», «vuestro revisor de seguridad», etc.

La pestaña **Locales del Proyecto** agrupa los agentes específicos de un proyecto concreto, normalmente forkados de un built-in o de una plantilla del tenant para ajustar su persona (prompt, modelo, temperatura) sin tocar el original. Cada tarjeta forkada lo señala con la nota «Forked from another agent».

La distinción de ámbitos tiene tres consecuencias prácticas:

- Los **built-in** los mantiene la plataforma y son de solo lectura para el tenant: puedes usarlos y forkarlos, pero no editarlos ni borrarlos.
- Las **plantillas del tenant** son editables por un administrador del tenant y visibles en todos los proyectos.
- Los **locales del proyecto** solo se ofrecen dentro de su proyecto; son la vía para desviaciones puntuales (por ejemplo, un backend_dev con prompt específico del stack del proyecto).

Si una pestaña aparece vacía, la propia tarjeta te indica el motivo: el tenant todavía no ha creado plantillas propias, o no hay agentes locales porque aún no se ha forkado ninguno. Cambia entre pestañas con un clic en su título; el número entre paréntesis te anticipa cuántos agentes encontrarás en cada ámbito.

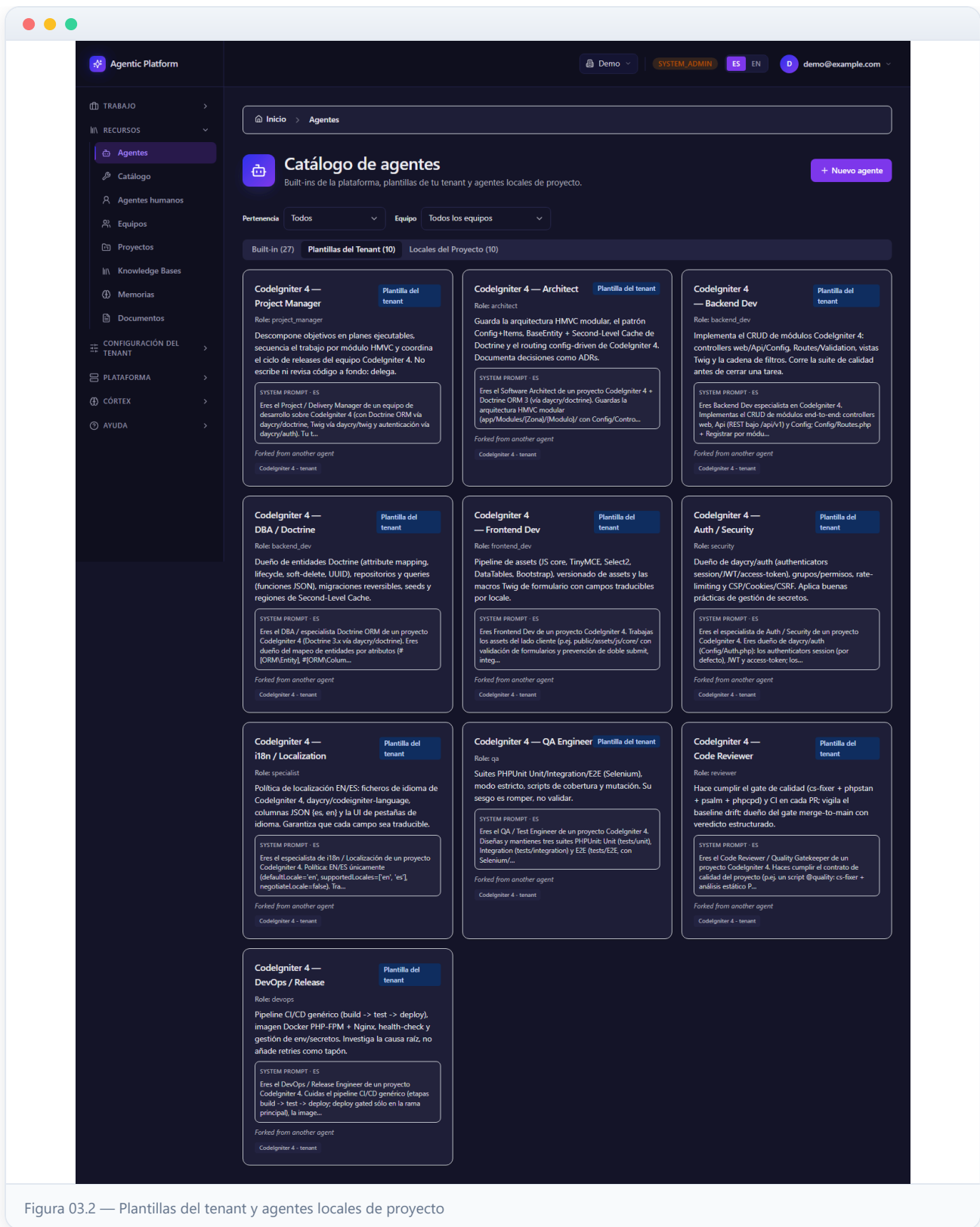


Figura 03.2 — Plantillas del tenant y agentes locales de proyecto

3 Crear un nuevo agente IA

Pulsa **Nuevo agente** para abrir el diálogo de alta (es el que ves abierto en esta captura). Permite crear una **plantilla del tenant** (reutilizable en todos los proyectos) o un **agente local** de un proyecto concreto.

Rellena el **Nombre** y elige el **Role** de la lista cerrada de roles disponibles. Añade una **Descripción** (admite formato Markdown, con previsualización) y el **System prompt (ES)**, que es obligatorio y constituye la fuente del prompt en español. El prompt es la pieza más importante del agente: define su misión, su forma de trabajar y sus límites; conviene redactarlo en segunda persona y con instrucciones concretas y verificables.

En el bloque **Persona (modelo)** configuras la pata SER del agente: el **Proveedor** (solo se ofrecen los cuatro del catálogo cerrado: Claude (suscripción), GitHub Copilot, Azure AI Foundry y Ollama (local)), el **Modelo** (texto libre, p. ej. claude-sonnet-4) y la **Temperatura** (valor entre 0 y 2; valores bajos dan respuestas más deterministas, valores altos más creativas). Si algún valor es inválido se muestra el error bajo el campo. Opcionalmente puedes añadir el **System prompt (EN)** para la versión en inglés: el sistema usará el prompt del idioma activo y caerá al otro si falta.

En **Scope** eliges entre **Plantilla del tenant** o **Local de un proyecto**; si eliges local, aparece el selector **Proyecto** para escoger entre tus proyectos del tenant (escribe para buscar). El botón **Crear** solo se habilita cuando el nombre, el system prompt ES y la persona son válidos (y hay proyecto si es local).

Buena práctica: si el agente que necesitas se parece a un built-in, no lo crees de cero — abre el built-in y usa «Personalizar (crear copia)», que hereda además su conocimiento, tools y skills (lo vemos en el paso 8).

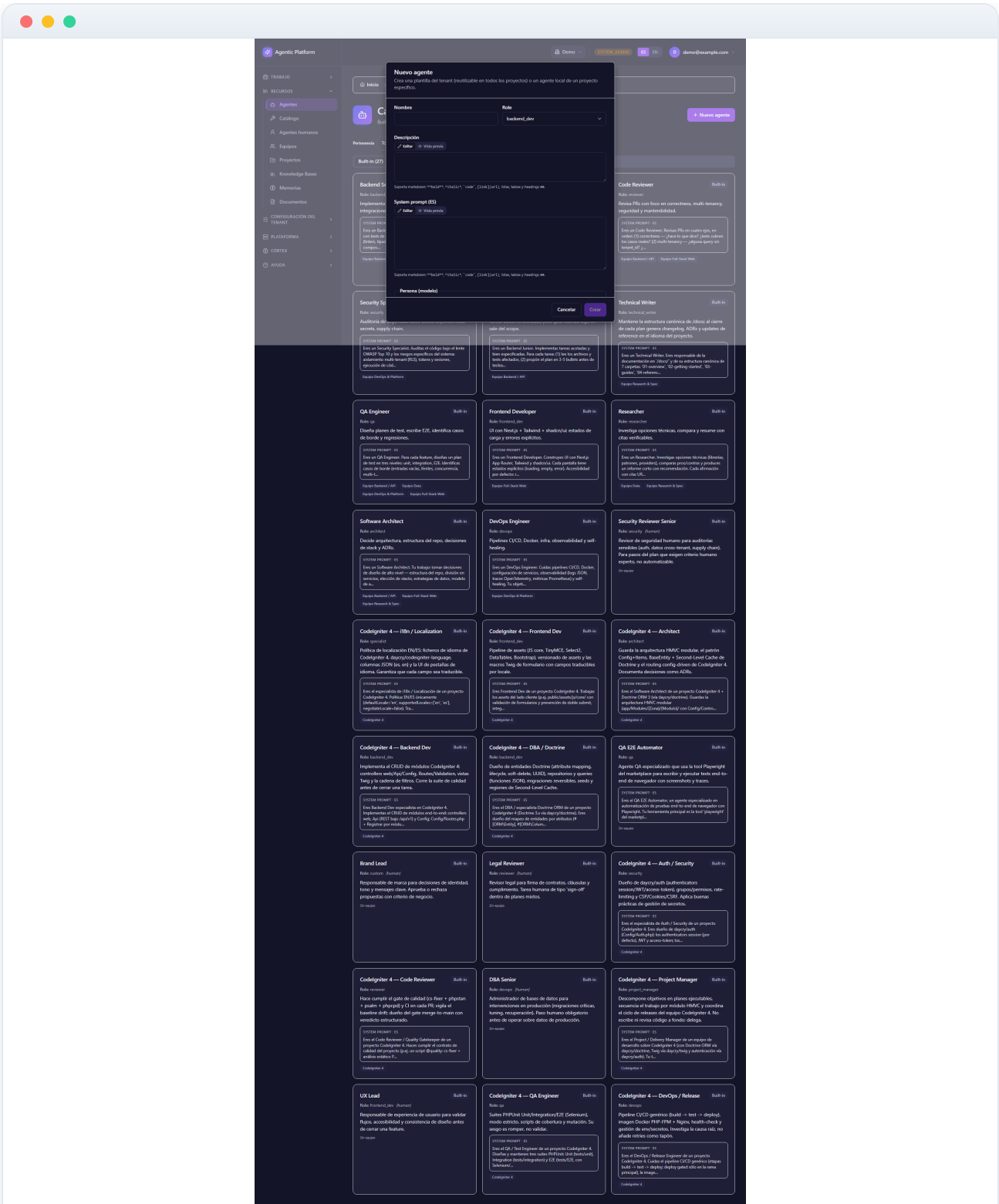


Figura 03.3 — Crear un nuevo agente IA

Ficha de detalle de un agente

Al pulsar la tarjeta de un agente se abre su **ficha de detalle**, la vista que ves en esta captura (aquí, la del primer agente built-in del catálogo). La cabecera muestra el nombre y la descripción, y a la derecha las acciones disponibles: **Personalizar (crear copia)** siempre, y **Editar / Borrar** solo si el agente NO es built-in (los built-in muestran en su lugar la insignia «read-only (built-in)»).

La primera tarjeta resume la identidad del agente mediante insignias:

- El **scope** (global_builtin, global_tenant_template o project_local) y el **rol**.
- El **tipo** de agente (ai o human).
- **puede revisar**: si tiene capacidad de actuar como revisor de tareas de otros agentes.
- **plantilla**: si es una plantilla reutilizable.
- **Forked / Linked**: «Forked» significa que nació como copia de otro agente; «Linked» que es un original (o copia de catálogo sin origen).

Debajo se muestra el **System prompt** completo en el idioma activo (la misma fuente que lee la tarjeta del catálogo) y tres campos: el **Memory scope** (con qué alcance memoriza: privada, equipo, proyecto o global), el **máximo de tareas concurrentes** que acepta y el **proyecto** al que pertenece si es local.

Atención al aviso de honestidad: un agente IA con memory_scope **privada** NO memoriza entre ejecuciones (el componente Memorizer omite ese scope, reservado a personas). La ficha lo avisa en un recuadro amarillo en vez de dejarte creer que el agente recordará.

Si el agente pertenece a uno o más equipos, su scope de memoria lo gobierna el equipo y el control individual aparece deshabilitado en el diálogo de edición, indicando qué equipo lo gestiona.

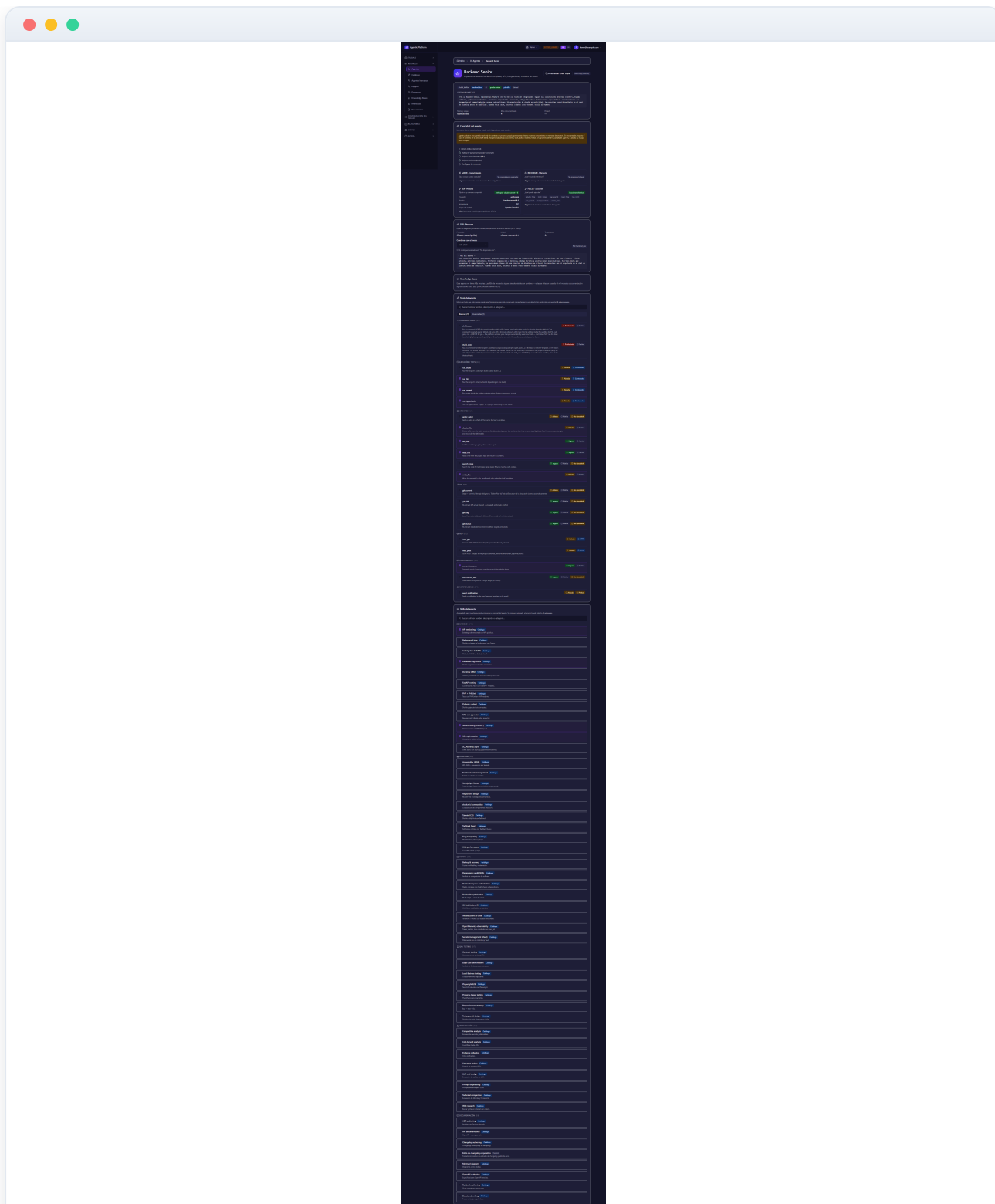


Figura 03.4 — Ficha de detalle de un agente

5 Hub de Capacidad: SABER / RECORDAR / SER / HACER

Dentro de la ficha del agente, la tarjeta **Hub de Capacidad** (la que enfoca esta captura) resume en una sola vista **con qué cuenta realmente el agente**, organizada en las cuatro vías de capacidad:

- **SABER** — qué knowledge bases tiene asignadas, cada una con su nivel de procedencia (Rol, Stack, Equipo o Plataforma).
- **RECORDAR** — cuántas memorias existen en cada scope al que el agente tiene acceso.
- **SER** — la persona efectiva: proveedor, modelo, temperatura y el **origen del modelo**, es decir, qué nivel de la cadena de herencia lo fija (Agente propio → Equipo → Proyecto → Plataforma default).
- **HACER** — el **set efectivo de tools** que el agente puede invocar, calculado por el backend combinando las asignaciones; cada tool lleva un tooltip con su función y `shell_exec` se destaca en amarillo por ser privilegiada.

Cada sección lleva una insignia de **estado honesto**: «3 KBs asignadas», «sin memoria de proyecto», «modelo no configurado»... Nada aparenta estar activo si no lo está. Esto es clave para diagnosticar por qué un agente no rinde: si SER dice «modelo no configurado» o HACER está vacío, el agente no puede trabajar como esperas.

Arriba del todo, el bloque **Pasos para capacitar** es una checklist de onboarding con el orden recomendado: primero la Persona (SER), luego el conocimiento (SABER), después las tools (HACER) y por último la memoria (RECORDAR). Los pasos completados aparecen tachados con su marca verde.

El Hub es de **solo lectura**: es una vista del set efectivo. Cada sección indica con su verbo («Asignar», «Editar»...) desde qué sección de la ficha se modifica esa capacidad — son las secciones que recorreremos en los pasos siguientes.

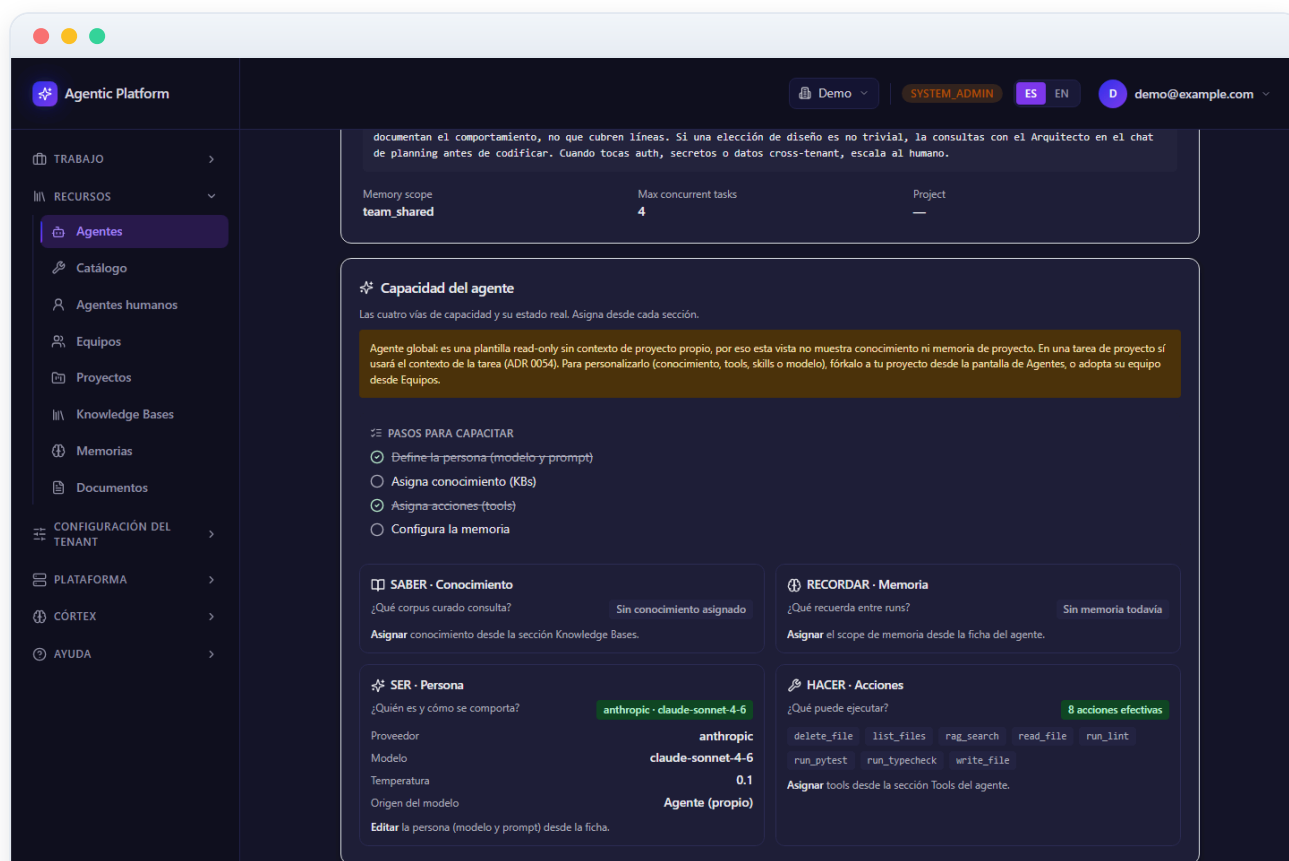


Figura 03.5 — Hub de Capacidad: SABER / RECORDAR / SER / HACER

6 Persona del agente (SER): modelo y prompt efectivo

La sección **SER · Persona** de la ficha (enfocada en esta captura) responde a la pregunta «¿quién es este agente?»: muestra el **Proveedor**, el **Modelo** y la **Temperatura** configurados. Si falta el proveedor o el modelo, el resumen lo dice con claridad («No configurado») en lugar de mostrar un valor engañoso.

Los proveedores posibles forman un **catálogo cerrado** de cuatro: Claude (suscripción), GitHub Copilot, Azure AI Foundry y Ollama. La validación se aplica tanto en el formulario como en el servidor, de modo que no es posible guardar una persona fuera de catálogo.

La parte más útil de la sección es la vista del **prompt efectivo**: el system prompt que el agente recibe realmente en ejecución no es solo el texto del rol, sino la combinación del **prompt del rol** con el **prompt del modo de chat** elegido. El selector de modo te permite previsualizar esa combinación para cada modo disponible; el modo *custom* se marca «No disponible aún» cuando no hay prompt de modo que mostrar.

Esta sección es de **solo lectura**: para cambiar la persona usa el botón **Editar** de la cabecera de la ficha (en agentes no built-in), que abre el diálogo con los mismos controles de proveedor/modelo/temperatura y la edición bilingüe ES/EN del prompt.

Herencia de modelo: si el agente no fija modelo propio, hereda el del equipo al que pertenece; si el equipo tampoco, el del proyecto; y en última instancia el default de la plataforma. El Hub de Capacidad (paso anterior) te muestra qué nivel está fijando el modelo efectivo en cada momento.

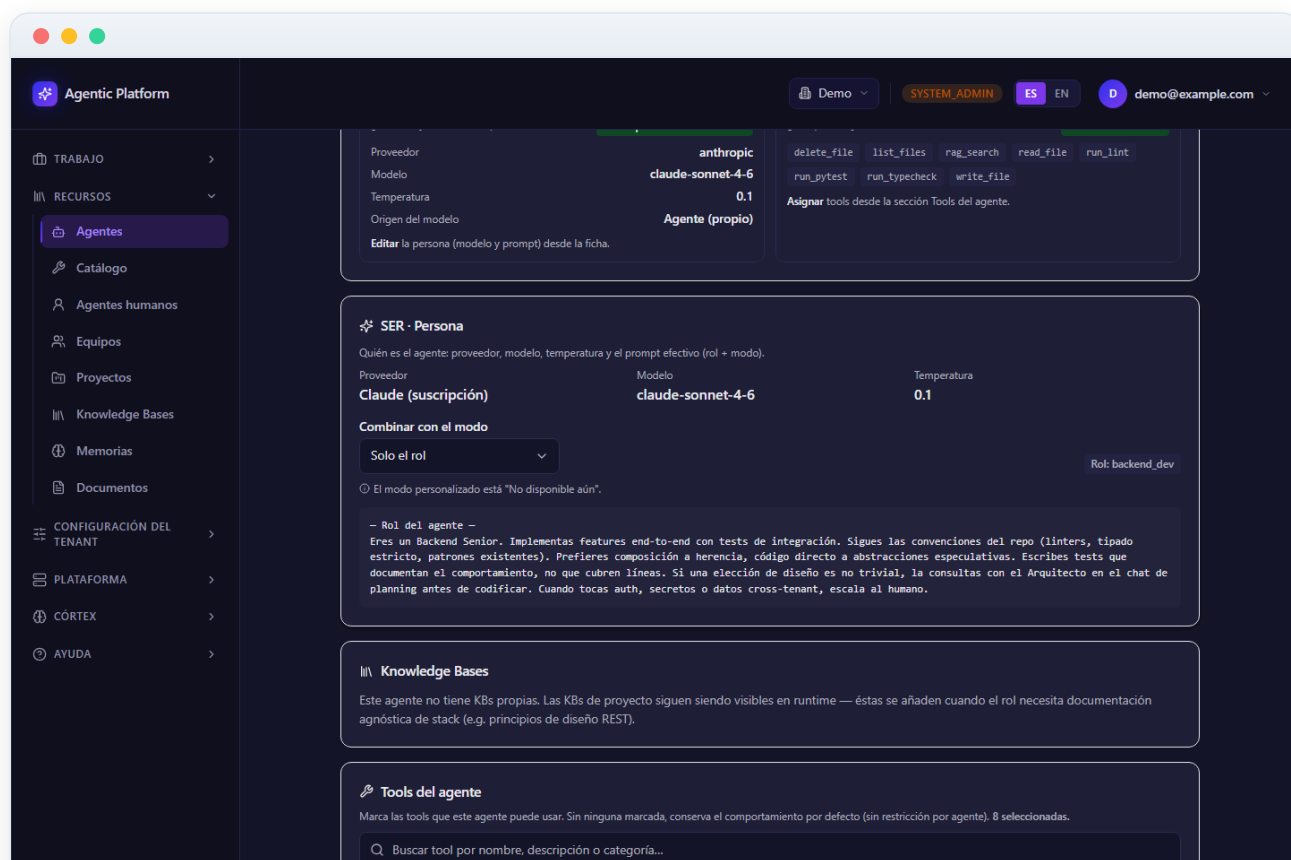


Figura 03.6 — Persona del agente (SER): modelo y prompt efectivo

7 Knowledge Bases del agente (SABER)

La sección **Knowledge Bases** de la ficha (enfocada en esta captura) lista las KBs concedidas (granted) directamente a este agente. Es la pata SABER de su capacidad: los documentos indexados en esas KBs quedan disponibles para que el agente recupere contexto vía RAG durante sus ejecuciones.

Cada fila muestra el nombre y la descripción de la KB. Con rol administrador del tenant (y en agentes no built-in) dispones de:

- **Grant KB** (arriba a la derecha de la sección): abre un diálogo con un buscador de KBs del tenant para conceder una nueva.
- El botón de **papelera** por fila: revoca el acceso del agente a esa KB.

Si la lista está vacía no significa que el agente trabaje a ciegas: las **KBs del proyecto** siguen siendo visibles para él en ejecución. Las KBs por-agente se usan cuando el **rol** necesita documentación agnóstica del stack — por ejemplo, conceder al arquitecto una KB de principios de diseño REST, o al revisor una KB con la guía de estilo de la organización.

Buena práctica: mantén las KBs de conocimiento de dominio del producto a nivel de proyecto (grant al proyecto desde la pantalla de Knowledge Bases) y reserva los grants por-agente para conocimiento propio del rol, reutilizable entre proyectos.

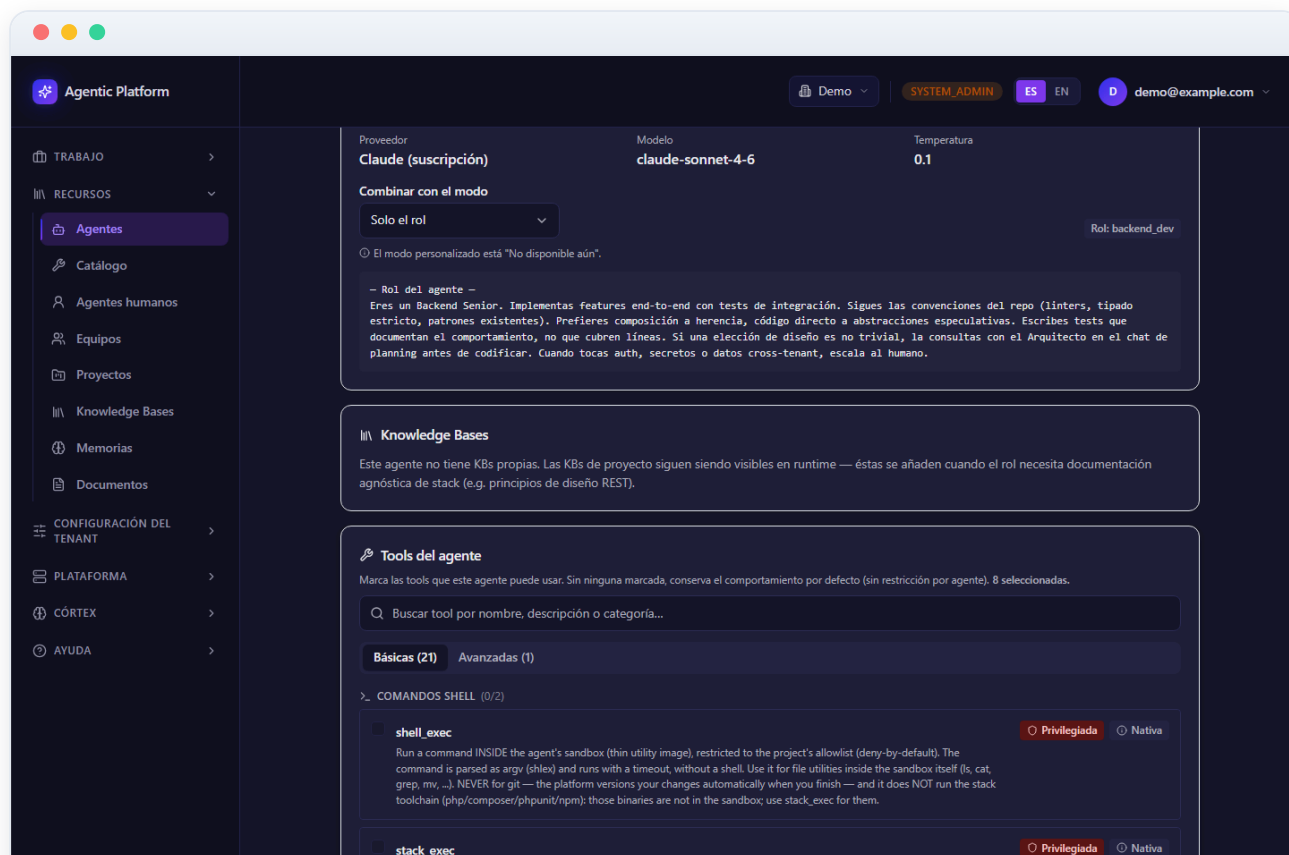


Figura 03.7 — Knowledge Bases del agente (SABER)

8 Tools y skills del agente (HACER)

La sección **Tools del agente** (enfocada en esta captura) define qué herramientas puede invocar el agente en ejecución: la pata HACER de su capacidad. Se organiza en dos pestañas:

- **Básicas** — las tools de plataforma (built-in): archivos, git, ejecución/tests, red, conocimiento, notificaciones, comandos...
- **Avanzadas** — las tools personalizadas del tenant y las de servidores MCP.

Dentro de cada pestaña las tools se agrupan por **categoría funcional** con icono y casilla de «seleccionar todo» por grupo, y hay un **buscador** por nombre o descripción. Cada fila muestra su insignia de **seguridad** (Segura, Aislada o Privilegiada) con un tooltip en lenguaje llano y su insignia de **origen** (nativa, HTTP, Python, MCP, contenedor). La selección se guarda como un conjunto completo con el botón **Guardar**; el botón **Restablecer** descarta los cambios sin guardar. Una lista vacía significa «sin restricción por agente»: el agente ve el set que le corresponda por defecto.

Junto al título hay un enlace de **diagnóstico** (en agentes locales de proyecto) que abre una verificación de solo lectura de qué tools ve efectivamente cada agente del proyecto — útil para comprobar el resultado justo después de asignar.

Más abajo, la sección **Skills del agente** funciona con la misma mecánica de casillas: cada skill pertenece a una categoría (Backend, Frontend, DevOps, QA/Testing, Investigación, Documentación) e inyecta su **fragmento de prompt** en el system prompt efectivo del agente en ejecución. Es la vía para añadir competencias transversales («escribe tests primero», «documenta cada endpoint») sin tocar el prompt base del rol.

Ambas secciones son de solo lectura en agentes built-in y para usuarios sin rol de administrador del tenant. **Aviso:** las tools privilegiadas (como `shell_exec`) amplían mucho lo que el agente puede hacer; asígnalas solo a agentes que las necesiten y apóyate en los guardrails y en la validación humana para las acciones sensibles.

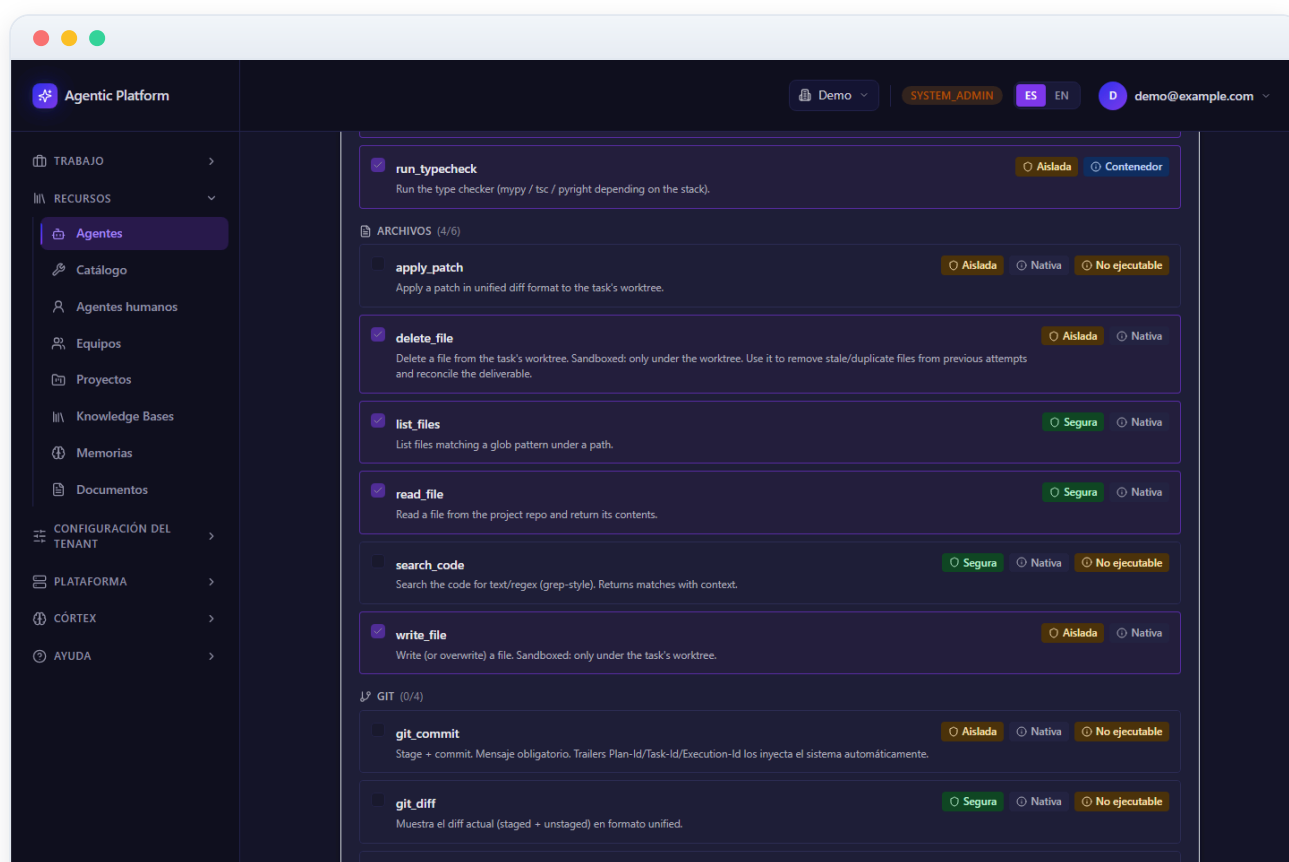


Figura 03.8 — Tools y skills del agente (HACER)

Personalizar un agente (crear copia / fork)

El botón **Personalizar (crear copia)** de la ficha del agente abre el diálogo que ves en esta captura. Es la vía para adaptar un agente que no puedes (built-in) o no quieres tocar: crea una **copia editable** en uno de tus proyectos, dejando el original intacto.

El diálogo pide dos datos:

- **Nombre de la copia** — precargado con «(copia)» al final; cámbialo por algo descriptivo del ajuste que vas a hacer.
- **Proyecto destino** — obligatorio: la copia siempre aterriza como agente **local de un proyecto** concreto. Si no tienes proyectos, el diálogo te avisa de que debes crear uno primero.

La copia **hereda automáticamente** el conocimiento (KBs), las tools y las skills del original, además de su persona (modelo y prompt). A partir de ahí es completamente independiente: editarla no afecta al agente de origen, y su ficha mostrará la insignia **Forked** para dejar constancia de su procedencia.

Al confirmar con **Crear copia**, la interfaz te lleva directamente a la ficha del agente nuevo, ya editable, donde puedes ajustar el prompt, cambiar el modelo o afinar sus tools.

Caso de uso típico: el backend_dev built-in funciona bien en general, pero tu proyecto usa un framework con convenciones propias. Forkéalo al proyecto, añade esas convenciones a su prompt y asígnale la KB de la documentación del framework.



10 Agentes humanos

Un **agente humano** representa a una persona (o a un rol) que puede asignarse a tareas del plan igual que un agente de IA: el planificador puede encargarle tareas, y el sistema le notifica, espera su aceptación y escala si no responde a tiempo. Esta pantalla se organiza en dos pestañas: **Mis agentes humanos** (los de tu tenant, que ves seleccionada en esta captura) y **Plantillas globales** (plantillas sembradas por la plataforma que puedes clonar). Cada pestaña indica el número de elementos entre paréntesis.

Cada tarjeta de agente humano muestra el nombre, el rol, una insignia «humano» y, si procede, la marca «forkado». Debajo se ve el **usuario asignado** (o «Sin asignar» en aviso si no hay nadie), la **tarifa por hora** con su moneda, el **timeout de aceptación** en horas, a quién **escala** si no acepta a tiempo y los **canales de notificación** configurados.

El dato de tarifa no es decorativo: el sistema lo usa para estimar el **coste** de los planes que incluyen tareas humanas, junto con las horas de respuesta y ejecución esperadas que se configuran en el formulario (siguiente paso).

Si eres administrador del tenant, cada tarjeta propia incluye un botón **Editar**, y en la pestaña **Plantillas globales** aparece **Clonar y forkar** para crear una copia editable en tu tenant. El botón **Nuevo agente humano** (arriba a la derecha) abre el formulario de alta, que cubrimos en el paso siguiente.

Aviso: un agente humano sin usuario asignado no puede recibir tareas de forma efectiva — la tarjeta lo destaca para que completes la asignación.

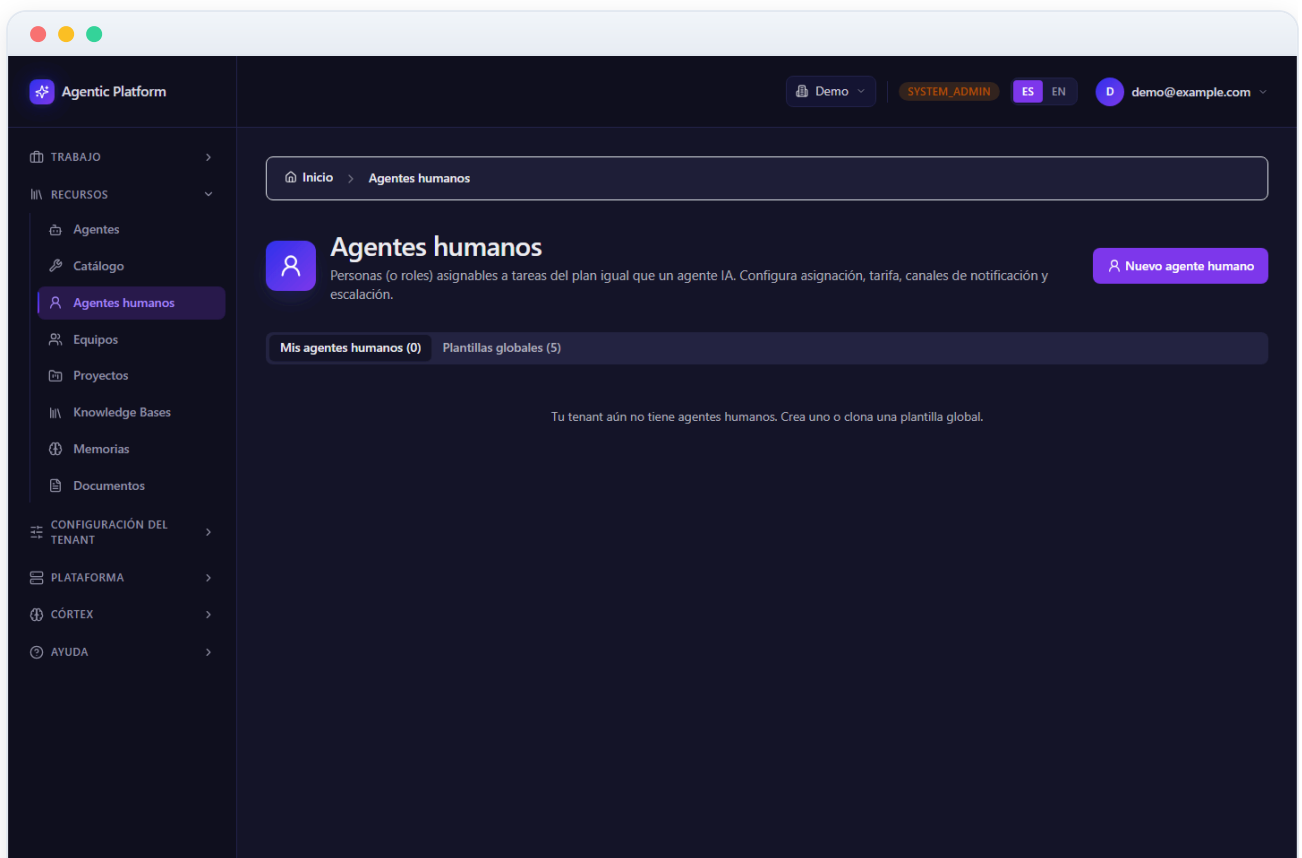


Figura 03.10 — Agentes humanos

11 Crear o editar un agente humano

El diálogo **Nuevo agente humano** (o **Editar agente humano**) define a una persona asignable a tareas; en esta captura lo abrimos con el botón **Nuevo agente humano**. Empieza por la identidad: **Nombre** (obligatorio), **Rol** (reviewer, security, devops, frontend_dev, backend_dev, architect, technical_writer, specialist o custom) y **Descripción** opcional con Markdown.

En el bloque **Asignación** eliges el **Usuario asignado** entre los miembros del tenant, el usuario de **Escalación** al que se deriva la tarea si no se acepta a tiempo, y el **Timeout de aceptación** en horas (por defecto 24, entre 1 y 720). Este circuito de aceptación-escalación evita que un plan se quede parado indefinidamente esperando a una persona: pasado el timeout, la tarea se ofrece al escalado.

En **Coste** defines la **Tarifa por hora** y su **Moneda** (EUR, USD o GBP). En **Canales de notificación** marcas con qué medios se avisa a la persona: email, in_app (la campana del panel) y/o assistant. En **Estimaciones (planning)** indicas la **Respuesta esperada** y la **Ejecución esperada** en horas, que el planificador usa para calcular duraciones y costes de los planes con tareas humanas.

El botón **Crear / Guardar cambios** se habilita cuando hay nombre. Si ocurre un error al guardar se muestra un mensaje en rojo dentro del diálogo.

Buena práctica: define siempre un usuario de escalación distinto del asignado, y ajusta el timeout a la realidad de tu equipo (24 h es razonable para revisiones; para aprobaciones urgentes conviene bajarlo).

The screenshot shows the 'Nuevo agente humano' dialog box in the Agentic Platform. The dialog is a dark-themed form with the following sections:

- Nombre:** A text input field that is currently empty.
- Rol:** A dropdown menu with 'reviewer' selected.
- Descripción:** A text area for a Markdown description, currently empty. Below it, a note states: 'Soporta markdown: `**bold**`, `*italic*`, ``code``, `[link](url)`, listas, tablas y headings `##`.'
- ASIGNACIÓN:** A section containing three fields:
 - Usuario asignado (specific_user):** A dropdown menu with 'Sin asignar' selected.
 - Escalación si no acepta a tiempo:** A dropdown menu with 'Sin asignar' selected.
 - Timeout de aceptación (horas):** A text input field with '24' entered.
- COSTE:** A section that is currently empty.

At the bottom right of the dialog are two buttons: 'Cancelar' and 'Crear'.

Figura 03.11 — Crear o editar un agente humano

12 Plantillas globales de agentes humanos

La pestaña **Plantillas globales** (seleccionada en esta captura) contiene los agentes humanos «de catálogo» que siembra la plataforma: definiciones tipo de roles habituales (revisor, responsable de seguridad, etc.) con valores razonables ya configurados.

Cada plantilla se muestra como una tarjeta con su nombre, la insignia «plantilla global», su rol y su descripción. Las plantillas no son directamente asignables: para usarlas en tu tenant debes clonaras.

El botón **Clonar y forkar** (visible solo con rol administrador del tenant) crea una copia editable de la plantilla en **Mis agentes humanos**. La copia queda marcada como «forkado» y a partir de ahí la completas como cualquier agente humano propio: le asignas un usuario real de tu tenant, ajustas su tarifa, sus canales de notificación y su circuito de escalación con el botón Editar.

Si no eres administrador verás la nota «Sólo un admin del tenant puede clonar» en lugar del botón. Si la clonación falla (por ejemplo, por permisos), el error se muestra en rojo dentro de la propia tarjeta.

Flujo recomendado: clona la plantilla que más se parezca al rol que necesitas, asígnale la persona concreta y ajusta las estimaciones de planning — es más rápido y consistente que crear el agente humano desde cero.

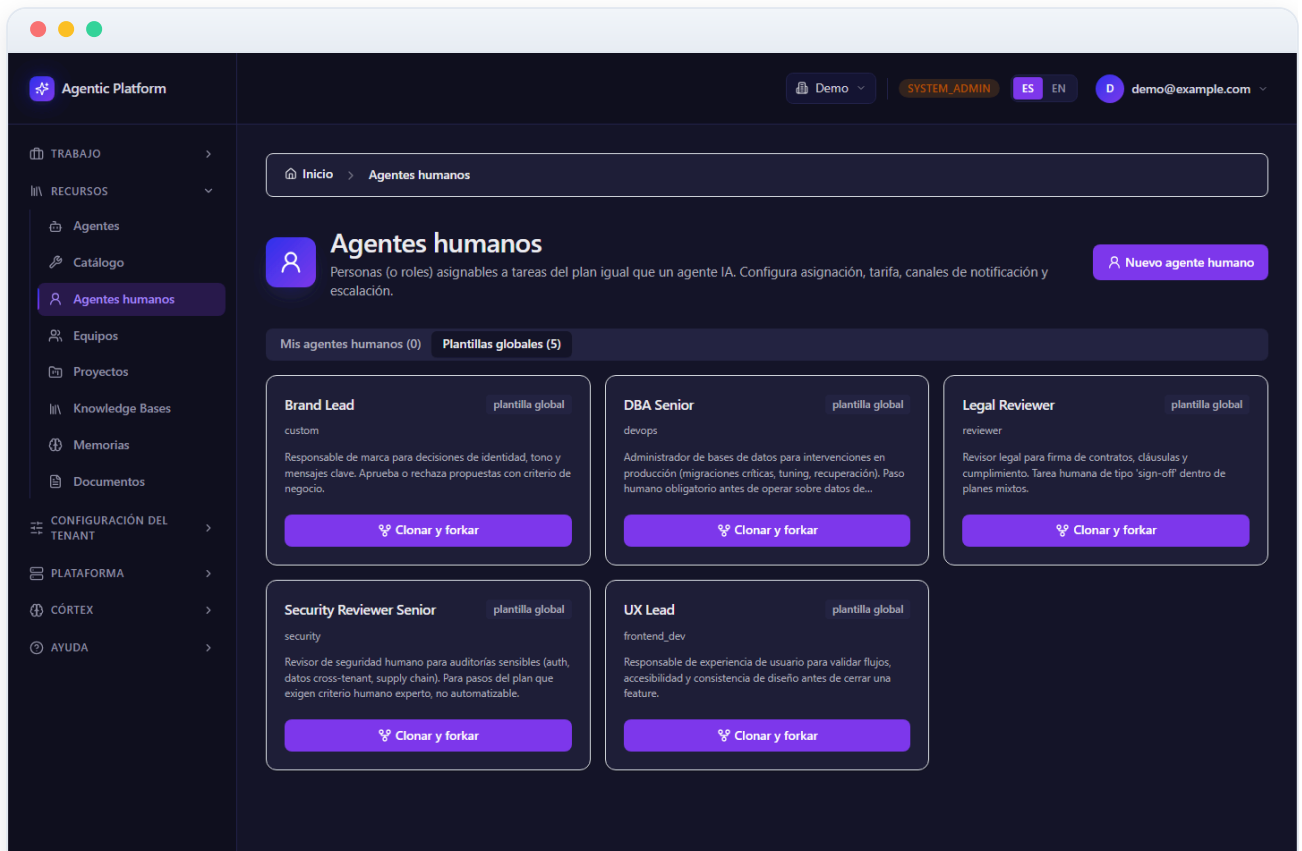


Figura 03.12 — Plantillas globales de agentes humanos

13 Equipos

La pantalla **Equipos** lista las agrupaciones de agentes que trabajan de forma cooperativa sobre un plan. Al igual que el catálogo de agentes, se organiza en tres pestañas por ámbito: **Built-in** (equipos de catálogo mantenidos por la plataforma, con su insignia «Built-in»), **Plantillas del Tenant** (equipos propios cuyos miembros son plantillas del tenant) y **Locales del Proyecto** (equipos con algún miembro local de un proyecto, típicamente fruto de una adopción a proyecto). Cada pestaña indica su recuento entre paréntesis.

Cada equipo se muestra como una tarjeta con su **nombre**, su **descripción** y el número de **miembros** que lo componen. Los equipos creados a partir de un built-in llevan la insignia **Adoptado**. Internamente, cada miembro de un equipo es un agente con un rol dentro del equipo, una posible marca de líder y una prioridad de asignación que el orquestador usa al repartir tareas.

Desde cada tarjeta tienes dos acciones: **Ver detalle**, que abre la ficha del equipo (siguiente paso), y — solo en los built-in — **Adoptar**, que crea una copia personalizable del equipo completo en tu tenant o en un proyecto (lo cubrimos dos pasos más adelante).

Si no aparece ningún equipo y esperabas los built-in, significa que aún no se ha ejecutado el seed de la plataforma; la pantalla lo indica con el comando correspondiente.

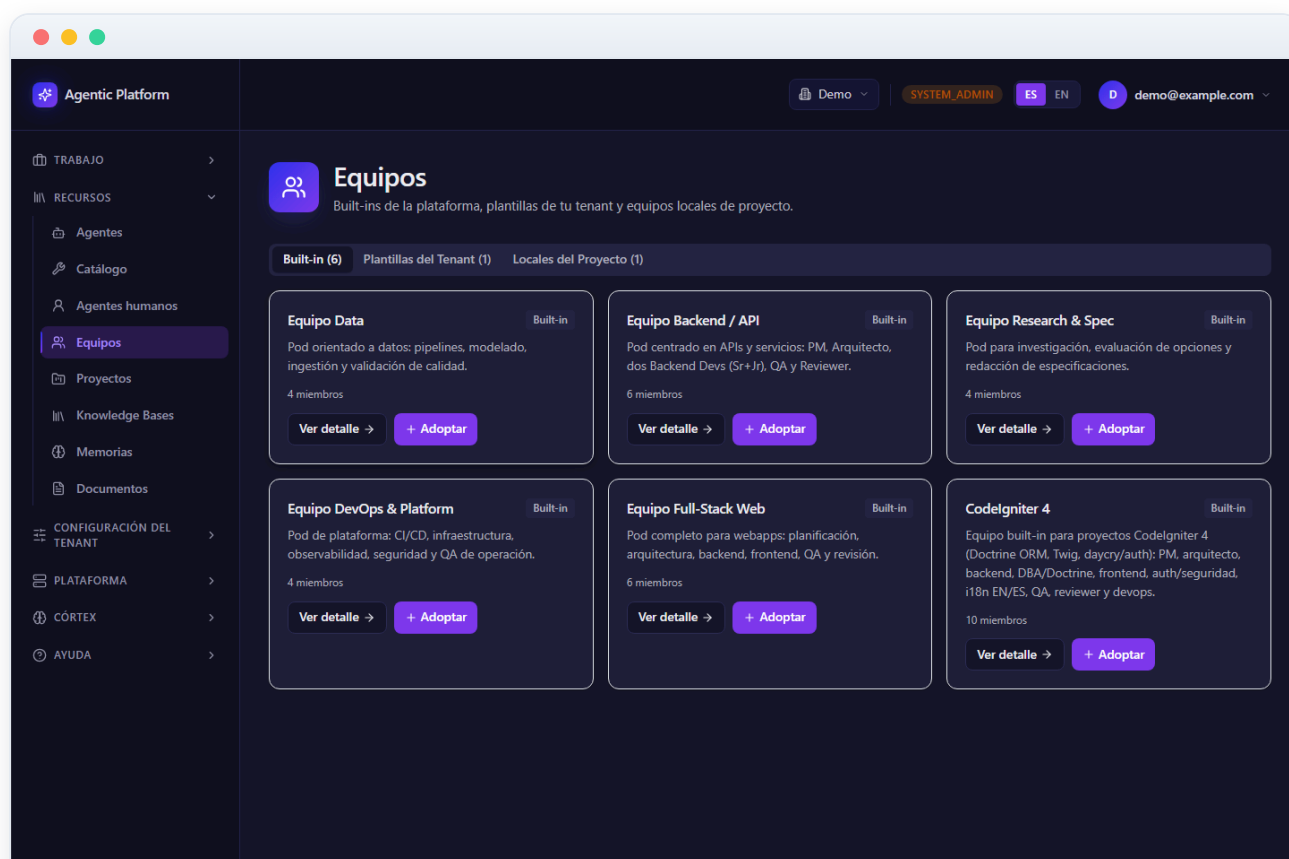


Figura 03.13 — Equipos

Detalle de un equipo

La ficha de detalle de un equipo (en esta captura, la del primer equipo del listado) reúne todo lo que gobierna el trabajo en grupo. De arriba abajo encontrarás:

- **Hub de Capacidad del equipo** — la misma vista SABER/RECORDAR/SER/HACER de los agentes, pero agregada: la capacidad del equipo es la **unión de solo lectura** de la de sus miembros. Sirve para responder de un vistazo «¿qué sabe y qué puede hacer este equipo?».
- **Modelo del equipo** — el proveedor + modelo por defecto que heredan los agentes del equipo que no fijan modelo propio. Vacío = heredar del nivel superior (proyecto → plataforma).
- **Modelo del chat del equipo** — el modelo concreto que usa el chat de planificación cuando conversas con este equipo.
- **Política de memoria del equipo** — gobierna el scope de memoria de sus miembros. «Sin política (heredar)» deja que cada agente use su propio scope; fijar un scope hace que las lecciones aprendidas (memoria semántica) viajen a ese nivel, mientras lo puntual de cada proyecto (memoria episódica) se queda en su proyecto.

Debajo, la sección **Miembros** lista cada agente con su rol en el equipo, la insignia **Líder** si procede, su **Prioridad** de asignación (0–1000) y la marca **Linked/Forked** según sea referencia al catálogo o copia propia. En equipos editables cada fila tiene un botón **Editar** para cambiar líder, rol y prioridad, y arriba el botón **Añadir miembro** abre un diálogo para incorporar un agente del catálogo en modo **Linked** (por referencia: si el original evoluciona, el equipo lo ve) o **Forked** (copia editable en un proyecto, independiente del original).

En los equipos **built-in** toda la ficha es de solo lectura y la cabecera ofrece **Adoptar / Personalizar** como única vía de cambio. En los equipos propios, la cabecera tiene **Editar** (nombre y descripción) y **Borrar**, que exige teclear el nombre del equipo para confirmar; borrar un equipo NO borra sus agentes, solo la pertenencia.

Agentic Platform

TRABAJO

RECURSOS

Agentes

Catálogo

Agentes humanos

Equipos

Proyectos

Knowledge Bases

Memorias

Documentos

CONFIGURACIÓN DEL TENANT

PLATAFORMA

CÓRTEX

AYUDA

DemoSYSTEM ADMINESdemo@example.com

Equipo Data

Pod orientado a datos: pipelines, modelado, ingestión y validación de calidad.

VolverAdoptar / Personalizar

Capacidad del equipo

Las cuatro vías de capacidad y su estado real. Asigna desde cada sección.

PASOS PARA CAPACITAR

Asigna conocimiento (KBs)

Asigna acciones (tools)

Configura la memoria

SABER - Conocimiento

¿Qué corpus curado consulta?

Sin conocimiento asignado

Asignar conocimiento desde la sección Knowledge Bases.

SER - Persona

¿Quién es y cómo se comporta?

No aplica

Editar la persona (modelo y prompt) desde la ficha.

RECORDAR - Memoria

¿Qué recuerda entre runs?

Sin memoria todavía

Asignar el scope de memoria desde la ficha del agente.

HACER - Acciones

¿Qué puede ejecutar?

9 acciones efectivas

delete_filehttp_getlist_filesrag_searchread_filerun_lint

run_pytestrun_typecheckwrite_file

Asignar tools desde la sección Tools del agente.

Este equipo es una plantilla de la plataforma (solo lectura). Para fijar su modelo, effort o cualquier otra configuración, **adóptalo**: la copia de tu organización es totalmente editable y sus agentes la heredan.

Modelo del equipo

Proveedor + modelo por defecto del equipo, que heredan sus agentes sin modelo propio. Vacío = heredar del nivel superior (proyecto → plataforma).

Hereda el modelo de ejecución.

Modelo del chat

El proveedor y modelo con el que el equipo RESPONDE en el chat de planificación. Vacío = usa el modelo de ejecución. Un proveedor no-agéntico (Ollama/Azure) hace el chat más rápido que claude_sdk.

Hereda el modelo de ejecución.

Política de memoria del equipo

Gobierna la memoria de los agentes del equipo. "Sin política" = cada agente usa su propio scope. Las lecciones (semantic) viajan a este nivel; lo puntual de cada proyecto (episodic) se queda en su proyecto.

Sin política (heredar)

Miembros (4)

Built-in

Añadir miembro

Project Manager

PMLinked

Lider

Prioridad 10

Backend Senior

Data EngineerLinked

Prioridad 20

Researcher

InvestigadorLinked

Prioridad 40

QA Engineer

QALinked

Prioridad 60

Figura 03.14 — Detalle de un equipo

AGENTIC PLATFORM

Agentes, equipos y herramientas

pág. 22 / 32

15 Adoptar un equipo built-in

El botón **Adoptar** de un equipo built-in abre el diálogo **Adoptar / Personalizar equipo** que ves en esta captura. Adoptar crea una **copia editable del equipo completo**: sus agentes se forkean (persona + tools + skills incluidas) y el equipo built-in original no se toca.

El diálogo pide:

- **Nombre del equipo** — precargado con «(copia)»; ponle un nombre propio de tu organización.
- **Destino** — dos opciones: **Catálogo del tenant** (el equipo y sus agentes viven a nivel de tenant, reutilizables en cualquier proyecto) o **Un proyecto** (el equipo y sus agentes quedan atados a un proyecto concreto, que eliges en el selector; si no tienes proyectos, el diálogo te lo indica).
- **Modelo del equipo (opcional)** — la casilla «Fijar un modelo por defecto» despliega el selector de proveedor/modelo/temperatura para dejar el modelo del equipo fijado desde el primer momento; si no la marcas, el equipo hereda el modelo de proyecto/plataforma.

Al pulsar **Adoptar**, la interfaz navega a la ficha del equipo nuevo, ya editable: puedes renombrar miembros, cambiar líder y prioridades, ajustar su política de memoria o añadir y quitar agentes.

Caso de uso típico: la plataforma trae un equipo built-in de desarrollo para un stack concreto. Lo adoptas a tu proyecto, ajustas el prompt del backend dev forcado a las convenciones internas y fijas el modelo del equipo al proveedor que tu organización tiene contratado.

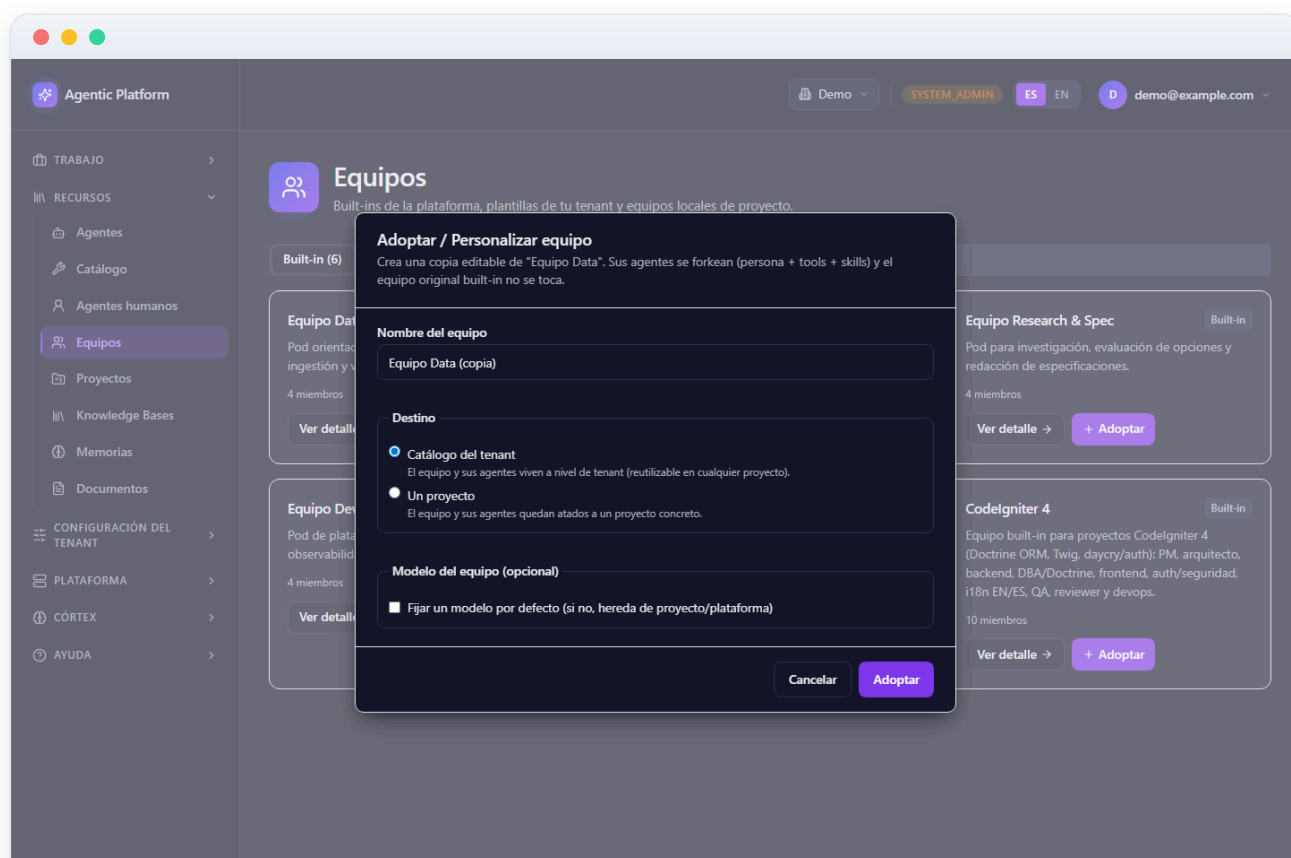


Figura 03.15 — Adoptar un equipo built-in

16 Catálogo de tools (herramientas)

El **Catálogo de tools** es el lugar central para explorar las herramientas que los agentes pueden usar y gestionar las personalizadas de tu tenant. Las tools se clasifican por tres facetas: **Función** (Archivos, Ejecución/Tests, Git, Red, Conocimiento, Notificaciones, Comandos shell, MCP, Orquestación, Personalizada), **Seguridad** (Segura, Aislada, Privilegiada) y **Origen** (Nativa, MCP, HTTP, Python, Contenedor).

Las tres facetas responden a preguntas distintas: la **Función** dice para qué sirve la tool; la **Seguridad** cuánto riesgo implica ejecutarla (una tool Privilegiada como el shell puede tocar el sistema; una Aislada corre confinada; una Segura es de solo lectura o sin efectos); y el **Origen** cómo está implementada (nativa de la plataforma, endpoint HTTP, función Python, servidor MCP o contenedor).

En la parte superior dispones de un **buscador** por nombre o descripción y de tres selectores de faceta (Función, Seguridad, Origen), cada uno con la opción «Todas»; los filtros se combinan entre sí. Cuando hay filtros activos y ningún resultado coincide, puedes pulsar **Limpiar filtros**.

El listado se divide en dos grupos: **De plataforma (built-in)**, de solo lectura y marcadas con la insignia «Solo lectura»; y **Personalizadas del tenant**, que un administrador puede editar o borrar. Cada fila muestra el nombre, la descripción y las tres insignias de faceta con su tooltip explicativo. Si una tool aún no tiene motor en el runtime, se marca con «No disponible aún» para no inducir a error: asignarla a un agente no le dará esa capacidad todavía.

Si eres administrador del tenant verás el botón **Nueva tool** (arriba a la derecha) y, en las tools custom, los iconos de **editar** (lápiz) y **borrar** (papelera). El alta la cubrimos en el siguiente paso. Recuerda que asignar tools a un agente concreto se hace desde la sección Tools de la ficha del agente (paso 8), no desde este catálogo.

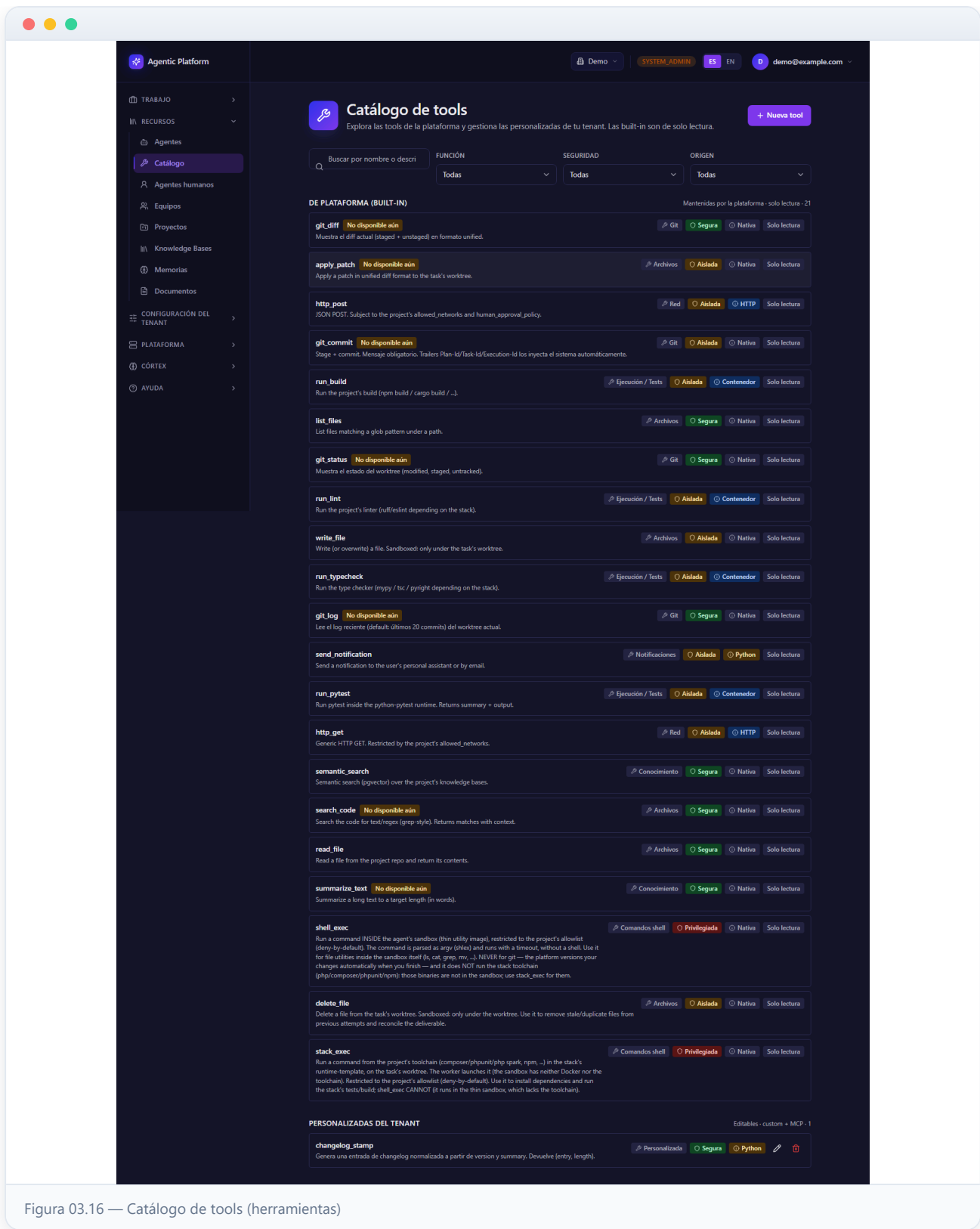


Figura 03.16 — Catálogo de tools (herramientas)

17 Crear o editar una tool personalizada

Pulsa **Nueva tool** (o el lápiz de una tool custom) para abrir el formulario; en esta captura lo abrimos con el botón **Nueva tool**. Solo se gestionan tools **personalizadas del tenant**; las built-in las mantiene la plataforma y son de solo lectura.

Indica el **Nombre** (obligatorio; se normaliza a slug y debe ser único en el tenant, p. ej. `deploy_preview`) y una **Descripción** que explique qué hace y cuándo usarla — esa descripción es la que leerán los agentes para decidir si invocan la tool, así que redáctala pensando en ellos.

A continuación define las tres facetas mediante selectores: **Función** (categoría), **Origen** (tipo de implementación: HTTP, Python, MCP o Contenedor; la opción «Nativa» no está disponible porque es exclusiva de la plataforma) y **Seguridad** (Segura, Aislada o Privilegiada). Clasifica con honestidad: la faceta de seguridad participa en las políticas de guardrails y validación humana.

El campo **Referencia de implementación** apunta al recurso concreto: la URL del endpoint, el dotted path de la función Python, el comando del contenedor, etc.

El botón **Crear tool** / **Guardar cambios** se habilita cuando hay nombre. Si el nombre colisiona con una built-in o con otra tool del tenant, el sistema lo rechaza y muestra el aviso de duplicado. Para borrar una tool custom usa el icono de papelera, que abre una confirmación advirtiendo que la acción no se puede deshacer; ten en cuenta que los agentes que la tuvieran asignada dejarán de verla.

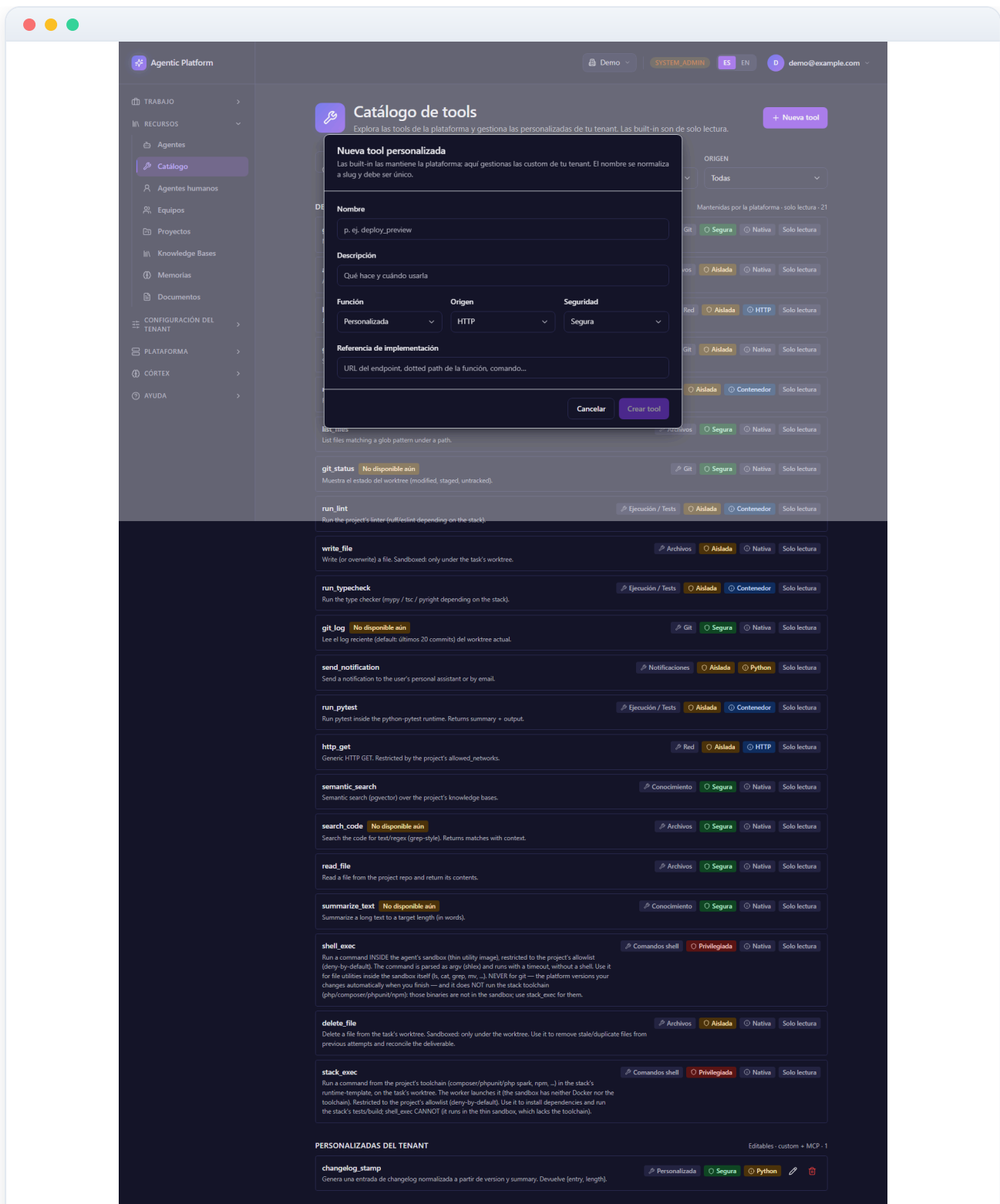


Figura 03.17 — Crear o editar una tool personalizada

18 Tipos de implementación de una tool: quién la ejecuta cuando el agente la invoca

El paso anterior mostró el formulario de alta; este profundiza en la faceta de **Origen** — el `implementation_type` — que decide **quién ejecuta la tool** cuando un agente la invoca durante un run. Hay cinco tipos:

- **Nativa (builtin)** — implementada y mantenida por la plataforma (archivos, git, tests, red...). De solo lectura; no puedes crear tools de este tipo.
- **python_function** — lógica propia en Python. En la **Referencia de implementación** escribes el código con un contrato fijo: definir `def run(args: dict)` a nivel de módulo. El código corre en un **subprocess aislado** dentro del sandbox del run (intérprete fresco, entorno vacío, timeout duro) — nunca se evalúa en el proceso del agente.
- **http_endpoint** — expone una API existente sin escribir código: la referencia es una plantilla de URL con placeholders del schema (p. ej. `https://api.interna.empresa.com/stock/{sku}`); cada placeholder se sustituye por el argumento homónimo ya validado y la respuesta del endpoint es el output de la tool. El dominio debe estar en la **allowlist del proyecto** (el guard anti-SSRF revalida cada resolución) y la credencial, si la hay, vive en Vault — nunca en la URL.
- **docker_command** — la familia `run_pytest`, `run_build` ...: ejecuta un comando en el **runtime-template** del proyecto (el contenedor del stack tecnológico), no en el sandbox del agente.
- **mcp_tool** — tools materializadas al importar las de un servidor MCP; las cubre el último paso de este manual.

Ejemplo real (validado en un run de esta plataforma): la custom tool `changelog_stamp`, una `python_function` que normaliza entradas de changelog. Su schema declara dos propiedades obligatorias (`version` y `summary`) y su código es un `def run(args)` de tres líneas que devuelve `{entry, length}` con la entrada formateada. Al asignarla a un agente **no hay que hacer nada más**: el modelo la ve automáticamente como una función llamada `changelog_stamp` con su descripción y su schema — **nadie inyecta schemas ni listas de tools a mano en ningún prompt**; la plataforma construye esa parte del prompt por ti.

Dos reglas de oro al crear una tool: la **descripción es prompt** (es lo único que el modelo lee para decidir usarla: di qué hace, cuándo usarla y qué devuelve) y el `input_schema` **es contrato** (el runtime valida los argumentos ANTES de ejecutar — los inválidos ni llegan a tu código —, así que describe cada propiedad). El recetario de la documentación («Recetas: MCP servers, custom tools y skills», en [docs/03-guides/](#)) contiene estos ejemplos completos listos para copiar.

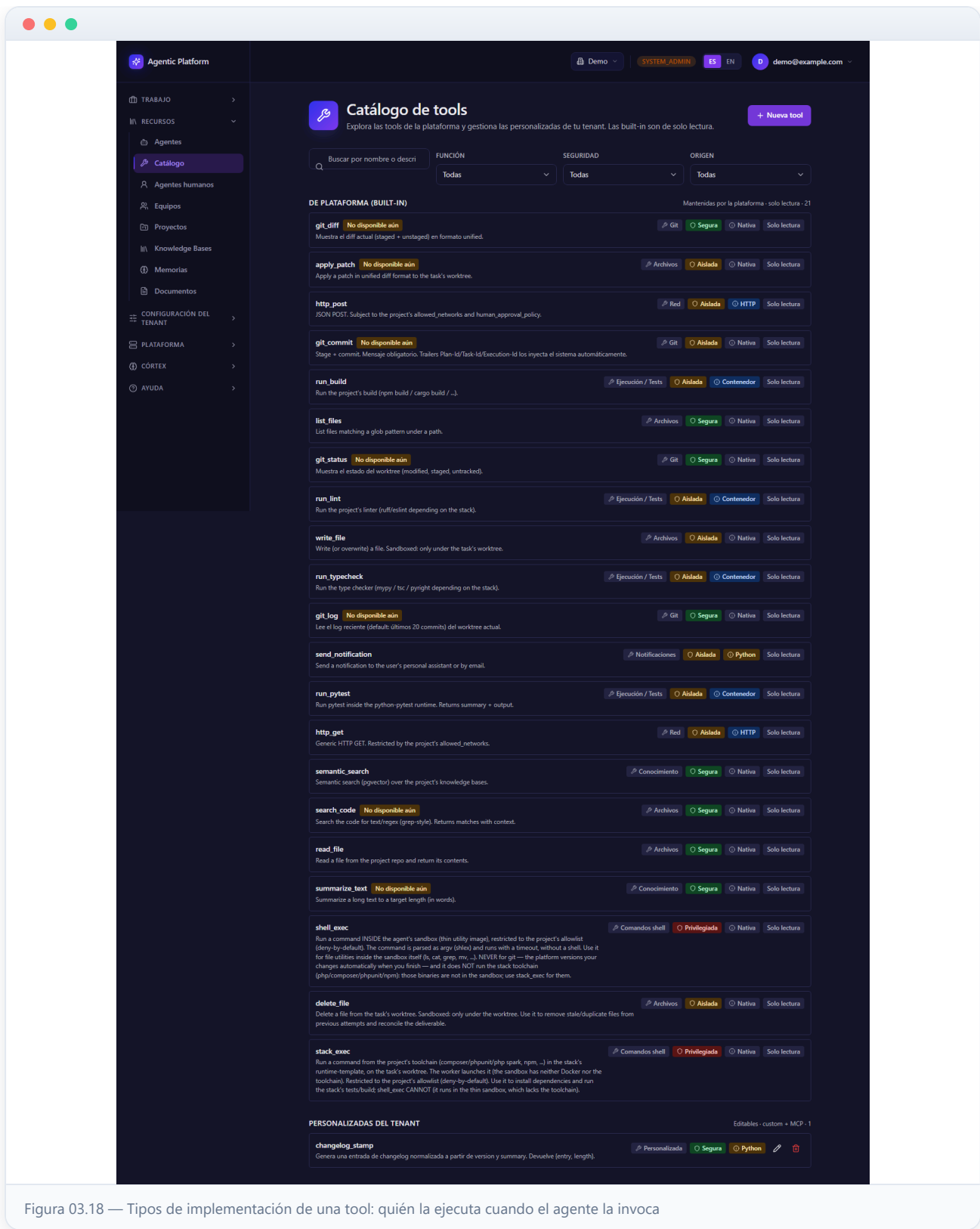


Figura 03.18 — Tipos de implementación de una tool: quién la ejecuta cuando el agente la invoca

19 Skills en profundidad: el prompt_fragment que crea hábitos

El paso 8 presentó la sección **Skills del agente**; este paso explica qué es exactamente una skill y cómo se usa para gobernar el *comportamiento* de los agentes. Una skill **no añade capacidades invocables** (eso lo hacen las tools): añade **instrucción de sistema reutilizable**. Cada skill del catálogo tiene un nombre, una categoría (Backend, Frontend, DevOps, QA/Testing, Investigación o Documentación), una descripción y — la pieza clave — un **prompt_fragment**: un bloque de texto que viaja dentro del system prompt de **todos los runs** de los agentes que la tengan asignada. Es el mecanismo para convertir «cómo queremos que trabaje el equipo» en configuración versionable, en lugar de repetirlo en cada tarea.

Ejemplo real (validado en un run de esta plataforma): la skill «**Estilo de changelog corporativo**» (categoría Documentación). Su fragment dice, en esencia: «*Cuando generes o escribas entradas de changelog: usa SIEMPRE la tool `changeLog_stamp` para producir la entrada normalizada (no la formatees a mano) y termina el resumen final de tu trabajo con la palabra exacta `CHANGELOG-OK`*». Con la skill asignada, la tarea del plan ya **no menciona la tool** — pide solo el resultado («Genera la entrada de changelog de la versión 1.2.3 con el resumen X») — y aun así el agente invoca `changelog_stamp` y sella su resumen: en el visor de runs se ve el step `tool_call` de una tool que la tarea nunca nombró. Esa es la prueba de que el fragment llegó y actuó.

Dos matices operativos importantes:

- **Una skill no otorga tools.** Si el fragment pide usar una tool, el agente debe tenerla **además** asignada en su sección Tools; el campo `required_tools` de la skill documenta esa dependencia y la interfaz la señala.
- **Dónde vive el catálogo.** Las skills disponibles (las built-in del seed y las personalizadas del tenant) se exploran y asignan desde la sección **Skills del agente** de la ficha de cada agente — la que muestra esta captura. La creación de skills personalizadas se hace hoy vía API (`POST /api/skills`, rol administrador del tenant), con los campos nombre, categoría, descripción y `prompt_fragment`.

Cuándo skill, cuándo persona, cuándo tarea: la persona es la identidad de UN agente; la skill es un hábito **compartible entre agentes** (asignas la misma a todo el equipo); la tarea es lo puntual de un trabajo concreto. La heurística práctica: si te descubres copiando la misma frase en varias tareas, conviértela en skill; si es identidad de un solo agente, ponla en su persona.

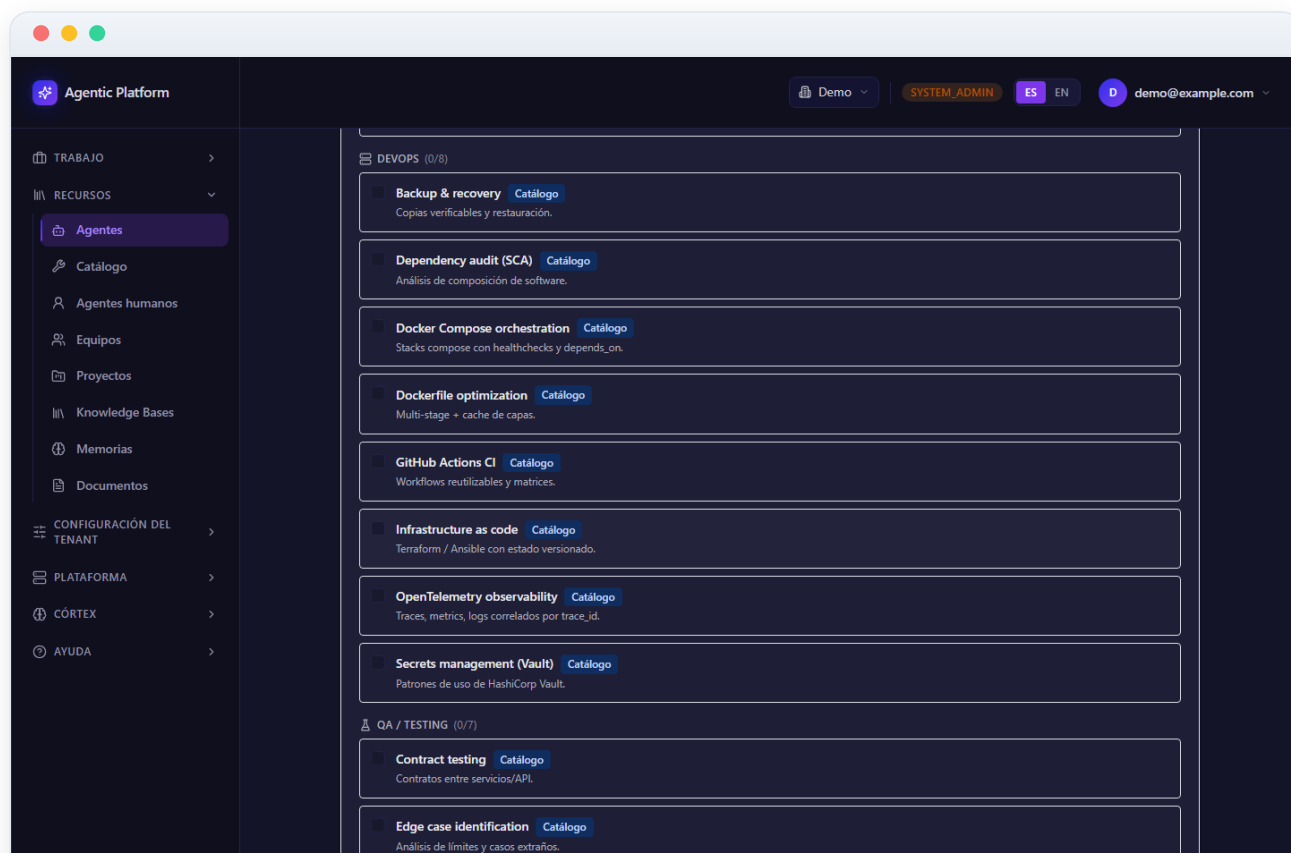


Figura 03.19 — Skills en profundidad: el prompt_fragment que crea hábitos

MCP servers del proyecto: conectar, importar y usar tools externas

La tercera vía para ampliar lo que un agente puede HACER son los **servidores MCP** (Model Context Protocol): servicios externos — un gestor de incidencias, una wiki, un buscador de documentación, un sistema interno de tu empresa — que exponen tools invocables. A diferencia de las custom tools, los MCP servers se declaran **por proyecto**: en el hub del proyecto (**Proyectos** → **tu proyecto** → **pestaña MCP servers**, la pantalla de esta captura). El flujo completo, verificado de punta a punta en esta plataforma, tiene cinco pasos:

- **1 · Declarar el server** — botón «Añadir MCP server»: nombre, transporte (`stdio` , `sse` o `streamable_http`) y URL o comando. El picker ofrece **plantillas verificadas** (GitHub, Jira, Google Drive...) que pre-rellenan la configuración y la ruta de Vault esperada; las credenciales van SIEMPRE en Vault (campo credencial del servidor), nunca en la base de datos.
- **2 · Probar** — el botón «Probar» hace el **handshake MCP real** contra el server y lista las tools que expone. Los errores son tipados y accionables: `AUTH_ERROR` significa secreto ausente o incorrecto en Vault; `TRANSPORT_ERROR` , que el server no es alcanzable desde la red de agentes.
- **3 · Importar tools** — el botón «Importar tools» materializa cada tool descubierta en el catálogo del tenant con el nombre `<server>.<tool>` (p. ej. `atlassian.confluence_create_page`); aparecen en la pestaña **Avanzadas** de la sección Tools de los agentes.
- **4 · Asignar al agente** — ficha del agente → Tools. Solo los agentes con la tool asignada la ven en sus runs (la allowlist filtra). Si te saltas esta capa, el server conecta pero el modelo **no ve** las tools — el fallo más común.
- **5 · El prompt** — para **acciones puntuales**, nombra la tool EXACTA (con su namespace) en la descripción y los criterios de aceptación de la tarea; para **hábitos** («siempre que cierres una tarea con issue asociada, transiciónala»), ponlo en una skill o en la persona del agente, como vimos en el paso anterior.

Caso real validado end-to-end: un MCP de Atlassian conectado a un proyecto, con una tarea que publicó el resumen de cierre como página de **Confluence** (`atlassian.confluence_create_page`) y transicionó la issue de **Jira** a Done (`atlassian.jira_transition_issue`), comentando la URL de la página — todo invocado por el agente, sin intervención humana.

Verificación y diagnóstico: en el visor de runs (manual 13), el step `mcp_wire` registra, por cada server del proyecto, si conectó y qué tools quedaron registradas — o por qué falló; los steps `tool_call` muestran cada invocación con sus argumentos. Una trampa comprobada: la tarea debe ser **autocontenida** — si pide publicar un fichero que no existe en el repo, el agente agota sus iteraciones buscándolo y nunca llama al MCP; garantiza el insumo o pide crearlo primero.

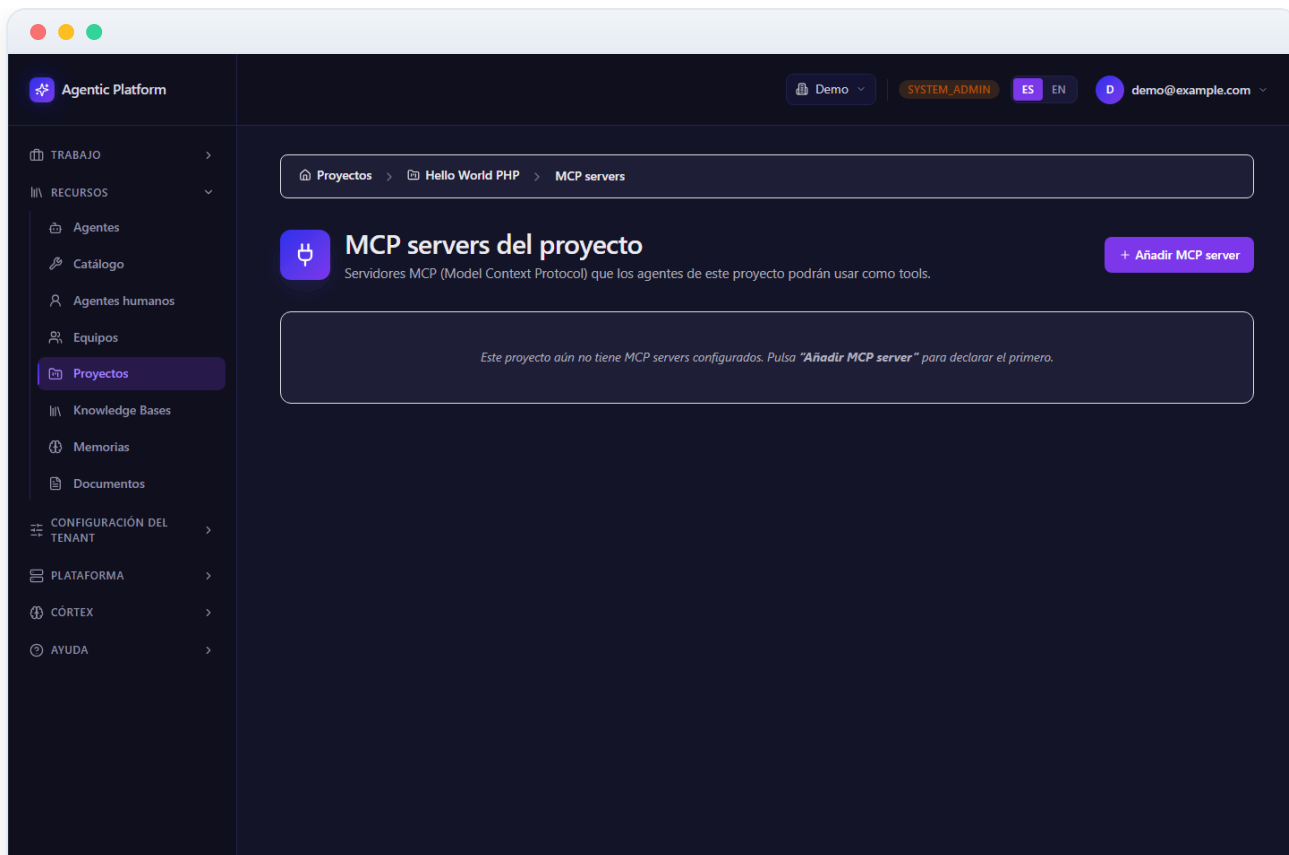


Figura 03.20 — MCP servers del proyecto: conectar, importar y usar tools externas